

webpack详解(2/4)

1. 课程目标

- 掌握webpack的实际开发应用；

2. 课程大纲

1. webpack原理；
2. webpack补充；

3. webpack原理

3.1. 工作原理概括

Webpack 以其使用简单著称，在使用它的过程中，使用者只需把它当作一个黑盒，需要关心的只有它暴露出来的配置。

3.1.1. 基本概念

- Entry：入口，Webpack 执行构建的第一步将从 Entry 开始，可抽象成输入；
- Module：模块，在 Webpack 里一切皆模块，一个模块对应着一个文件。Webpack 会从配置的 Entry 开始递归找出所有依赖的模块；
- Chunk：代码块，一个 Chunk 由多个模块组合而成，用于代码合并与分割；
- Loader：模块转换器，用于把模块原内容按照需求转换成新内容；
- Plugin：扩展插件，在 Webpack 构建流程中的特定时机会广播出对应的事件，插件可以监听这些事件的发生，在特定时机做对应的事情；

3.1.2. 流程概括

Webpack 的运行流程是一个串行的过程，从启动到结束会依次执行以下流程：

1. 初始化参数：从配置文件和 Shell 语句中读取与合并参数，得出最终的参数；

2. 开始编译：用上一步得到的参数初始化 Compiler 对象，加载所有配置的插件，执行对象的 run 方法开始执行编译；
3. 确定入口：根据配置中的 entry 找出所有的入口文件；
4. 编译模块：从入口文件出发，调用所有配置的 Loader 对模块进行翻译，再找出该模块依赖的模块，再递归本步骤直到所有入口依赖的文件都经过了本步骤的处理；
5. 完成模块编译：在经过第4步使用 Loader 翻译完所有模块后，得到了每个模块被翻译后的最终内容以及它们之间的依赖关系；
6. 输出资源：根据入口和模块之间的依赖关系，组装成一个个包含多个模块的 Chunk，再把每个 Chunk 转换成一个单独的文件加入到输出列表，这步是可以修改输出内容的最后机会；
7. 输出完成：在确定好输出内容后，根据配置确定输出的路径和文件名，把文件内容写入到文件系统。

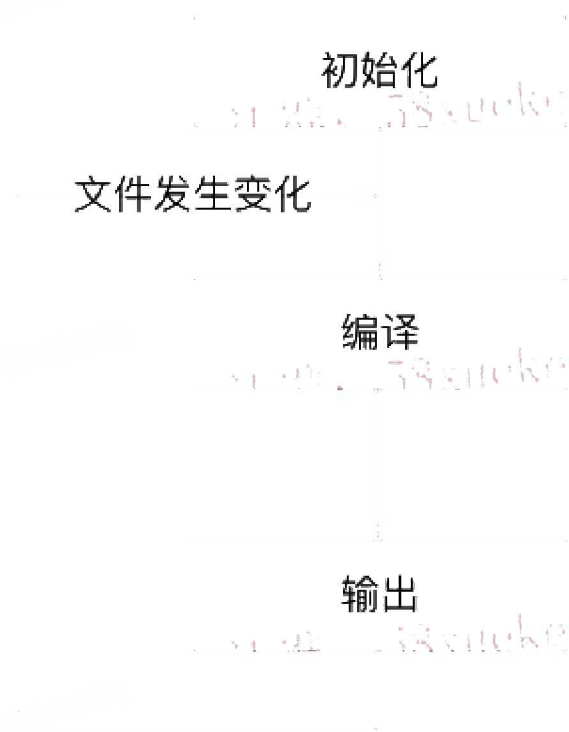
在以上过程中，Webpack 会在特定的时间点广播出特定的事件，插件在监听到感兴趣的事件后会执行特定的逻辑，并且插件可以调用 Webpack 提供的 API 改变 Webpack 的运行结果。

3.1.3. 流程细节

Webpack 的构建流程可以分为以下三大阶段：

1. 初始化：启动构建，读取与合并配置参数，加载 Plugin，实例化 Compiler；
2. 编译：从 Entry 发出，针对每个 Module 串行调用对应的 Loader 去翻译文件内容，再找到该 Module 依赖的 Module，递归地进行编译处理；
3. 输出：对编译后的 Module 组合成 Chunk，把 Chunk 转换成文件，输出到文件系统；

如果只执行一次构建，以上阶段将会按照顺序各执行一次。但在开启监听模式下，流程将变为如下：



在每个大阶段中又会发生很多事件，Webpack 会把这些事件广播出来供给 Plugin 使用。

3.1.3.1. 初始化阶段

事件名	解释
初始化参数	从配置文件和 Shell 语句中读取与合并参数，得出最终的参数。这个过程中还会执行配置文件中的插件实例化语句 <code>new Plugin()</code> 。
实例化 Compiler	用上一步得到的参数初始化 Compiler 实例，Compiler 负责文件监听和启动编译。Compiler 实例中包含了完整的 Webpack 配置，全局只有一个 Compiler 实例。
加载插件	依次调用插件的 <code>apply</code> 方法，让插件可以监听后续的所有事件节点。同时给插件传入 compiler 实例的引用，以方便插件通过 compiler 调用 Webpack 提供的 API。
environment	开始应用 Node.js 风格的文件系统到 compiler 对象，以方便后续的文件寻找和读取。
entry-option	读取配置的 Entrys，为每个 Entry 实例化一个对应的 EntryPlugin，为后面该 Entry 的递归解析工作做准备。
after-plugins	调用完所有内置的和配置的插件的 <code>apply</code> 方法。
after-resolvers	根据配置初始化完 resolver，resolver 负责在文件系统中寻找指定路径的文件。

3.1.3.2. 编译阶段

事件名	解释
run	启动一次新的编译。
watch-run	和 run 类似，区别在于它是在监听模式下启动的编译，在这个事件中可以获得是哪些文件发生了变化导致重新启动一次新的编译。
compile	该事件是为了告诉插件一次新的编译将要启动，同时会给插件带上 compiler 对象。
compilation	当 Webpack 以开发模式运行时，每当检测到文件变化，一次新的 Compilation 将被创建。一个 Compilation 对象包含了当前的模块资源、编译生成资源、变化的文件等。Compilation 对象也提供了很多事件回调供插件做扩展。
make	一个新的 Compilation 创建完毕，即将从 Entry 开始读取文件，根据文件类型和配置的 Loader 对文件进行编译，编译完后再找出该文件依赖的文件，递归的编译和解析。
after-compile	一次 Compilation 执行完成。
invalid	当遇到文件不存在、文件编译错误等异常时会触发该事件，该事件不会导致 Webpack 退出。

在编译阶段中，最重要的要数 compilation 事件了，因为在 compilation 阶段调用了 Loader 完成了每个模块的转换操作，在 compilation 阶段又包括很多小的事件，它们分别是：

事件名	解释
build-module	使用对应的 Loader 去转换一个模块。
normal-module-loader	在用 Loader 对一个模块转换完后，使用 acorn 解析转换后的内容，输出对应的抽象语法树（AST），以方便 Webpack 后面对代码的分析。
program	从配置的入口模块开始，分析其 AST，当遇到 require 等导入其它模块语句时，便将其加入到依赖的模块列表，同时对新找出的依赖模块递归分析，最终搞清所有模块的依赖关系。
seal	所有模块及其依赖的模块都通过 Loader 转换完成后，根据依赖关系开始生成 Chunk。

3.1.3.4. 输出阶段

事件名	解释
should-emit	所有需要输出的文件已经生成好，询问插件哪些文件需要输出，哪些不需要。
emit	确定好要输出哪些文件后，执行文件输出，可以在这里获取和修改输出内容。
after-emit	文件输出完毕。
done	成功完成一次完成的编译和输出流程。
failed	如果在编译和输出流程中遇到异常导致 Webpack 退出时，就会直接跳转到本步骤，插件可以在本事件中获取到具体的错误原因。

在输出阶段已经得到了各个模块经过转换后的结果和其依赖关系，并且把相关模块组合在一起形成一个 Chunk。在输出阶段会根据 Chunk 的类型，使用对应的模版生成最终要输出的文件内容。

3.2. 输出文件分析

Webpack 输出的 bundle.js 是什么样子的吗？为什么原来一个个的模块文件被合并成了一个单独的文件？为什么 bundle.js 能直接运行在浏览器中？本节将解释清楚以上问题。

先来看看由最初的例子中最简单的项目构建出的 bundle.js 文件内容，代码如下：

```

1  (
2      // webpackBootstrap 启动函数
3      // modules 即为存放所有模块的数组，数组中的每一个元素都是一个函数
4      function (modules) {
5          // 安装过的模块都存放在这里面
6          // 作用是把已经加载过的模块缓存在内存中，提升性能
7          var installedModules = {};
8
9          // 去数组中加载一个模块，moduleId 为要加载模块在数组中的 index
10         // 作用和 Node.js 中 require 语句相似
11         function __webpack_require__(moduleId) {
12             // 如果需要加载的模块已经被加载过，就直接从内存缓存中返回
13             if (installedModules[moduleId]) {
14                 return installedModules[moduleId].exports;
15             }
16
17             // 如果缓存中不存在需要加载的模块，就新建一个模块，并把它存在缓存中
18             var module = installedModules[moduleId] = {
19                 // 模块在数组中的 index
20                 i: moduleId,
21                 // 该模块是否已经加载完毕
22                 l: false,
23                 // 该模块的导出值
24                 exports: {}
25             };
26
27             // 从 modules 中获取 index 为 moduleId 的模块对应的函数
28             // 再调用这个函数，同时把函数需要的参数传入
29             modules[moduleId].call(module.exports, module, module.exports,
__webpack_require__);
30             // 把这个模块标记为已加载
31             module.l = true;
32             // 返回这个模块的导出值
33             return module.exports;
34         }
35
36         // Webpack 配置中的 publicPath，用于加载被分割出去的异步代码
37         __webpack_require__.p = "";
38
39         // 使用 __webpack_require__ 去加载 index 为 0 的模块，并且返回该模块导出
的内容
40         // index 为 0 的模块就是 main.js 对应的文件，也就是执行入口模块
41         // __webpack_require__.s 的含义是启动模块对应的 index
42         return __webpack_require__(__webpack_require__.s = 0);
43     }

```

```

44     })(
45
46     // 所有的模块都存放在了一个数组里，根据每个模块在数组的 index 来区分和定位模块
47     [
48         /* 0 */
49         (function (module, exports, __webpack_require__) {
50             // 通过 __webpack_require__ 规范导入 show 函数，show.js 对应的模块
51             index 为 1
52             const show = __webpack_require__(1);
53             // 执行 show 函数
54             show('Webpack');
55         }),
56         /* 1 */
57         (function (module, exports) {
58             function show(content) {
59                 window.document.getElementById('app').innerText = 'Hello,'
60                 + content;
61             }
62             // 通过 CommonJS 规范导出 show 函数
63             module.exports = show;
64         })
65     ]
66 );

```

以上看上去复杂的代码其实是一个立即执行函数，可以简写为如下：

```

1  (function(modules) {
2
3      // 模拟 require 语句
4      function __webpack_require__() {
5      }
6
7      // 执行存放所有模块数组中的第0个模块
8      __webpack_require__(0);
9
10 })([/*存放所有模块的数组*/])

```

 复制代码

bundle.js 能直接运行在浏览器中的原因在于输出的文件中通过 `__webpack_require__` 函数定义了一个可以在浏览器中执行的加载函数来模拟 Node.js 中的 `require` 语句。

原来一个个独立的模块文件被合并到了一个单独的 `bundle.js` 的原因在于浏览器不能像 Node.js 那样快速地去本地加载一个个模块文件，而必须通过网络请求去加载还未得到的文件。如果模块数量很多，加载时间会很长，因此把所有模块都存放在了数组中，执行一次网络加载。

如果仔细分析 `__webpack_require__` 函数的实现，你还有发现 Webpack 做了缓存优化：执行加载过的模块不会再执行第二次，执行结果会缓存在内存中，当某个模块第二次被访问时会直接去内存中读取被缓存的返回值。

3.2.1. 分割代码时的输出

在按需加载时的优化方案中，webapck的输出文件会发生变化：



复制代码

```
1 // 异步加载 show.js
2 import('./show').then((show) => {
3   // 执行 show 函数
4   show('Webpack');
5 });
```

重新构建后会输出两个文件，分别是执行入口文件 `bundle.js` 和 异步加载文件 `0.bundle.js`。

其中 `0.bundle.js` 内容如下：



复制代码

```
1 // 加载在本文件(0.bundle.js)中包含的模块
2 webpackJsonp(
3   // 在其它文件中存放着的模块的 ID
4   [0],
5   // 本文件所包含的模块
6   [
7     // show.js 所对应的模块
8     (function (module, exports) {
9       function show(content) {
10         window.document.getElementById('app').innerText = 'Hello, ' + conte
11         nt;
12       }
13       module.exports = show;
14     })
15   ]
16 );
```

`bundle.js` 内容如下：


```

1  (function (modules) {
2      /**
3       * webpackJsonp 用于从异步加载的文件中安装模块。
4       * 把 webpackJsonp 挂载到全局是为了方便在其它文件中调用。
5       *
6       * @param chunkIds 异步加载的文件中存放的需要安装的模块对应的 Chunk ID
7       * @param moreModules 异步加载的文件中存放的需要安装的模块列表
8       * @param executeModules 在异步加载的文件中存放的需要安装的模块都安装成功后，需要执行的模块对应的 index
9       */
10     window["webpackJsonp"] = function webpackJsonpCallback(chunkIds, moreModules, executeModules) {
11         // 把 moreModules 添加到 modules 对象中
12         // 把所有 chunkIds 对应的模块都标记成已经加载成功
13         var moduleId, chunkId, i = 0, resolves = [], result;
14         for (; i < chunkIds.length; i++) {
15             chunkId = chunkIds[i];
16             if (installedChunks[chunkId]) {
17                 resolves.push(installedChunks[chunkId][0]);
18             }
19             installedChunks[chunkId] = 0;
20         }
21         for (moduleId in moreModules) {
22             if (Object.prototype.hasOwnProperty.call(moreModules, moduleId)) {
23                 modules[moduleId] = moreModules[moduleId];
24             }
25         }
26         while (resolves.length) {
27             resolves.shift()();
28         }
29     };
30
31     // 缓存已经安装的模块
32     var installedModules = {};
33
34     // 存储每个 Chunk 的加载状态；
35     // 键为 Chunk 的 ID，值为0代表已经加载成功
36     var installedChunks = {
37         1: 0
38     };
39
40     // 模拟 require 语句，和上面介绍的一致
41     function __webpack_require__(moduleId) {
42         // ... 省略和上面一样的内容
43     }

```

```

44 =
45 /**
46  * 用于加载被分割出去的，需要异步加载的 Chunk 对应的文件
47  * @param chunkId 需要异步加载的 Chunk 对应的 ID
48  * @returns {Promise}
49  */
50 =
51 __webpack_require__.e = function requireEnsure(chunkId) {
52     // 从上面定义的 installedChunks 中获取 chunkId 对应的 Chunk 的加载状态
53     var installedChunkData = installedChunks[chunkId];
54     // 如果加载状态为0表示该 Chunk 已经加载成功了，直接返回 resolve Promise
55     if (installedChunkData === 0) {
56         return new Promise(function (resolve) {
57             resolve();
58         });
59     }
60     // installedChunkData 不为空且不为0表示该 Chunk 正在网络加载中
61     if (installedChunkData) {
62         // 返回存放在 installedChunkData 数组中的 Promise 对象
63         return installedChunkData[2];
64     }
65
66     // installedChunkData 为空，表示该 Chunk 还没有加载过，去加载该 Chunk 对应的
67 文件
68     var promise = new Promise(function (resolve, reject) {
69         installedChunkData = installedChunks[chunkId] = [resolve, reject];
70     });
71     installedChunkData[2] = promise;
72
73     // 通过 DOM 操作，往 HTML head 中插入一个 script 标签去异步加载 Chunk 对应
    的 JavaScript 文件
74     var head = document.getElementsByTagName('head')[0];
75     var script = document.createElement('script');
76     script.type = 'text/javascript';
77     script.charset = 'utf-8';
78     script.async = true;
79     script.timeout = 120000;
80
81     // 文件的路径为配置的 publicPath、chunkId 拼接而成
82     script.src = __webpack_require__.p + "" + chunkId + ".bundle.js";
83
84     // 设置异步加载的最长超时时间
85     var timeout = setTimeout(onScriptComplete, 120000);
86     script.onerror = script.onload = onScriptComplete;
87
88     // 在 script 加载和执行完成时回调
89     function onScriptComplete() {
90         // 防止内存泄露

```



```

90     script.onerror = script.onload = null;
91     clearTimeout(timeout);
92
93     // 去检查 chunkId 对应的 Chunk 是否安装成功, 安装成功时才会存在于 installed
    Chunks 中
94     var chunk = installedChunks[chunkId];
95     if (chunk !== 0) {
96         if (chunk) {
97             chunk[1](new Error('Loading chunk ' + chunkId + ' failed.'));
98         }
99         installedChunks[chunkId] = undefined;
100     }
101 };
102 head.appendChild(script);
103
104     return promise;
105 };
106
107 // 加载并执行入口模块, 和上面介绍的一致
108 return __webpack_require__(__webpack_require__.s = 0);
109 })
110 (
111     // 存放所有没有经过异步加载的, 随着执行入口文件加载的模块
112     [
113         // main.js 对应的模块
114         (function (module, exports, __webpack_require__) {
115             // 通过 __webpack_require__.e 去异步加载 show.js 对应的 Chunk
116             __webpack_require__.e(0).then(__webpack_require__.bind(null, 1)).th
117 en((show) => {
118             // 执行 show 函数
119             show('Webpack');
120         });
121     })
122 ];
123 );

```

这里的 bundle.js 和上面所讲的 bundle.js 非常相似, 区别在于:

- 多了一个 `__webpack_require__.e` 用于加载被分割出去的, 需要异步加载的 Chunk 对应的文件;
- 多了一个 `webpackJsonp` 函数用于从异步加载的文件中安装模块;

在使用了 `CommonsChunkPlugin` 去提取公共代码时输出的文件和使用异步加载时输出的文件是一样的, 都会有 `__webpack_require__.e` 和 `webpackJsonp`。原因在于提取公共代码和异步加载本质上都是代码分割。

3.3. 编写loader

Loader 就像是一个翻译员，能把源文件经过转化后输出新的结果，并且一个文件还可以链式的经过多个翻译员翻译。

以处理 SCSS 文件为例：

1. SCSS 源代码会先交给 `sass-loader` 把 SCSS 转换成 CSS；
2. 把 `sass-loader` 输出的 CSS 交给 `css-loader` 处理，找出 CSS 中依赖的资源、压缩 CSS 等；
3. 把 `css-loader` 输出的 CSS 交给 `style-loader` 处理，转换成通过脚本加载的 JavaScript 代码；

可以看出以上的处理过程需要有顺序的链式执行，先 `sass-loader` 再 `css-loader` 再 `style-loader`。以上处理的 Webpack 相关配置如下：

 复制代码

```
1 module.exports = {
2   module: {
3     rules: [
4       {
5         // 增加对 SCSS 文件的支持
6         test: /\.scss$/,
7         // SCSS 文件的处理顺序为先 sass-loader 再 css-loader 再 style-loader
8         use: [
9           'style-loader',
10          {
11            loader: 'css-loader',
12            // 给 css-loader 传入配置项
13            options: {
14              minimize: true,
15            }
16          },
17          'sass-loader'],
18        },
19      ]
20    },
21  };
```

3.3.1. Loader的职责

由上面的例子可以看出：一个 Loader 的职责是单一的，只需要完成一种转换。如果一个源文件需要经历多步转换才能正常使用，就通过多个 Loader 去转换。在调用多个 Loader 去转换一个文件时，每个

Loader 会链式的顺序执行，第一个 Loader 将会拿到需处理的原内容，上一个 Loader 处理后的结果会传给下一个接着处理，最后的 Loader 将处理后的最终结果返回给 Webpack。

所以，在你开发一个 Loader 时，请保持其职责的单一性，你只需关心输入和输出。

3.3.2. Loader的基础

由于 Webpack 是运行在 Node.js 之上的，一个 Loader 其实就是一个 Node.js 模块，这个模块需要导出一个函数。这个导出的函数的工作就是获得处理前的原内容，对原内容执行处理后，返回处理后的内容。

一个最简单的 Loader 的源码如下：



复制代码

```
1 module.exports = function(source) {  
2   // source 为 compiler 传递给 Loader 的一个文件的原内容  
3   // 该函数需要返回处理后的内容，这里简单起见，直接把原内容返回了，相当于该 Loader 没有  
   做任何转换  
4   return source;  
5 };
```

由于 Loader 运行在 Node.js 中，你可以调用任何 Node.js 自带的 API，或者安装第三方模块进行调用：



复制代码

```
1 const sass = require('node-sass');  
2 module.exports = function(source) {  
3   return sass(source);  
4 };
```

3.3.3. Loader的进阶

以上只是个最简单的 Loader，Webpack 还提供一些 API 供 Loader 调用，下面来一一介绍。

3.3.3.1. 获得 Loader 的 options

在最上面处理 SCSS 文件的 Webpack 配置中，给 css-loader 传了 options 参数，以控制 css-loader。如何在自己编写的 Loader 中获取到用户传入的 options 呢？需要这样做：

```

1  const loaderUtils = require('loader-utils');
2  module.exports = function(source) {
3    // 获取到用户给当前 Loader 传入的 options
4    const options = loaderUtils.getOptions(this);
5    return source;
6  };

```

3.3.3.2. 返回其他结果

上面的 Loader 都只是返回了原内容转换后的内容，但有些场景下还需要返回除了内容之外的东西。

例如以用 babel-loader 转换 ES6 代码为例，它还需要输出转换后的 ES5 代码对应的 Source Map，以方便调试源码。为了把 Source Map 也一起随着 ES5 代码返回给 Webpack，可以这样写：

```

1  module.exports = function(source) {
2    // 通过 this.callback 告诉 Webpack 返回的结果
3    this.callback(null, source, sourceMaps);
4    // 当你使用 this.callback 返回内容时，该 Loader 必须返回 undefined，
5    // 以让 Webpack 知道该 Loader 返回的结果在 this.callback 中，而不是 return 中
6    return;
7  };

```

其中的 this.callback 是 Webpack 给 Loader 注入的 API，以方便 Loader 和 Webpack 之间通信。

this.callback 的详细使用方法如下：

```

1  this.callback(
2    // 当无法转换原内容时，给 Webpack 返回一个 Error
3    err: Error | null,
4    // 原内容转换后的内容
5    content: string | Buffer,
6    // 用于把转换后的内容得出原内容的 Source Map，方便调试
7    sourceMap?: SourceMap,
8    // 如果本次转换为原内容生成了 AST 语法树，可以把这个 AST 返回，
9    // 以方便之后需要 AST 的 Loader 复用该 AST，以避免重复生成 AST，提升性能
10   abstractSyntaxTree?: AST
11 );

```

Source Map 的生成很耗时，通常在开发环境下才会生成 Source Map，其它环境下不用生成，以加速构建。为此 Webpack 为 Loader 提供了 this.sourceMap API 去告诉 Loader 当前构建环境下用户是

否需要 Source Map。如果你编写的 Loader 会生成 Source Map，请考虑到这点。

3.3.3.3. 同步与异步

Loader 有同步和异步之分，上面介绍的 Loader 都是同步的 Loader，因为它们的转换流程都是同步的，转换完成后再返回结果。但在有些场景下转换的步骤只能是异步完成的，例如你需要通过网络请求才能得出结果，如果采用同步的方式网络请求就会阻塞整个构建，导致构建非常缓慢。

在转换步骤是异步时，你可以这样：



复制代码

```
1 module.exports = function(source) {
2     // 告诉 Webpack 本次转换是异步的，Loader 会在 callback 中回调结果
3     var callback = this.async();
4     someAsyncOperation(source, function(err, result, sourceMaps, ast) {
5         // 通过 callback 返回异步执行后的结果
6         callback(err, result, sourceMaps, ast);
7     });
8 }
```

3.3.3.4. 处理二进制数据

在默认的情况下，Webpack 传给 Loader 的原内容都是 UTF-8 格式编码的字符串。但有些场景下 Loader 不是处理文本文件，而是处理二进制文件，例如 file-loader，就需要 Webpack 给 Loader 传入二进制格式的数据。为此，你需要这样编写 Loader：



复制代码

```
1 module.exports = function(source) {
2     // 在 exports.raw === true 时，Webpack 传给 Loader 的 source 是 Buffer 类
    型的
3     source instanceof Buffer === true;
4     // Loader 返回的类型也可以是 Buffer 类型的
5     // 在 exports.raw !== true 时，Loader 也可以返回 Buffer 类型的结果
6     return source;
7 };
8 // 通过 exports.raw 属性告诉 Webpack 该 Loader 是否需要二进制数据
9 module.exports.raw = true;
```

以上代码中最关键的代码是最后一行 `module.exports.raw = true;`，没有该行 Loader 只能拿到字符串。

3.3.3.5. 缓存加速

在有些情况下，有些转换操作需要大量计算非常耗时，如果每次构建都重新执行重复的转换操作，构建将会变得非常缓慢。为此，Webpack 会默认缓存所有 Loader 的处理结果，也就是说在需要被处理的文件或者其依赖的文件没有发生变化时，是不会重新调用对应的 Loader 去执行转换操作的。

如果你想让 Webpack 不缓存该 Loader 的处理结果，可以这样：



复制代码

```
1 module.exports = function(source) {  
2   // 关闭该 Loader 的缓存功能  
3   this.cacheable(false);  
4   return source;  
5 };
```

3.3.4. 其他Loader API

除了以上提到的在 Loader 中能调用的 Webpack API 外，还存在以下常用 API：

- this.context：当前处理文件的所在目录，假如当前 Loader 处理的文件是 /src/main.js，则 this.context 就等于 /src；
- this.resource：当前处理文件的完整请求路径，包括 querystring，例如 /src/main.js?name=1；
- this.resourcePath：当前处理文件的路径，例如 /src/main.js；
- this.resourceQuery：当前处理文件的 querystring；
- this.target：等于 Webpack 配置中的 Target；
- this.loadModule：当 Loader 在处理一个文件时，如果依赖其它文件的处理结果才能得出当前文件的结果时，就可以通过 this.loadModule(request: string, callback: function(err, source, sourceMap, module)) 去获得 request 对应文件的处理结果；
- this.resolve：像 require 语句一样获得指定文件的完整路径，使用方法为 resolve(context: string, request: string, callback: function(err, result: string))；
- this.addDependency：给当前处理文件添加其依赖的文件，以便再其依赖的文件发生变化时，会重新调用 Loader 处理该文件。使用方法为 addDependency(file: string)；
- this.addContextDependency：和 addDependency 类似，但 addContextDependency 是把整个目录加入到当前正在处理文件的依赖中。使用方法为 addContextDependency(directory: string)；
- this.clearDependencies：清除当前正在处理文件的所有依赖，使用方法为 clearDependencies()；
- this.emitFile：输出一个文件，使用方法为 emitFile(name: string, content: Buffer|string, sourceMap: {...})；

其它没有提到的 API 可以去 [Webpack 官网](#) 查看。

3.3.5. 加载本地Loader

在开发 Loader 的过程中，为了测试编写的 Loader 是否能正常工作，需要把它配置到 Webpack 中后，才可能会调用该 Loader。上文中使用的 Loader 都是通过 Npm 安装的，要使用 Loader 时会直接使用 Loader 的名称，代码如下：

```
1  module.exports = {  
2    module: {  
3      rules: [  
4        {  
5          test: /\.css$/,  
6          use: ['style-loader'],  
7        },  
8      ]  
9    },  
10  };
```

复制代码

如果还采取以上的方法去使用本地开发的 Loader 将会很麻烦，因为你需要确保编写的 Loader 的源码是在 node_modules 目录下。为此你需要先把编写的 Loader 发布到 Npm 仓库后再安装到本地项目使用。

解决以上问题的便捷方法有两种，分别如下：

3.3.5.1. npm link

Npm link 专门用于开发和调试本地 Npm 模块，能做到在不发布模块的情况下，把本地的一个正在开发的模块的源码链接到项目的 node_modules 目录下，让项目可以直接使用本地的 Npm 模块。由于是通过软链接的方式实现的，编辑了本地的 Npm 模块代码，在项目中也能使用到编辑后的代码。

完成 Npm link 的步骤如下：

1. 确保正在开发的本地 Npm 模块（也就是正在开发的 Loader）的 package.json 已经正确配置好；
2. 在本地 Npm 模块根目录下执行 npm link，把本地模块注册到全局；
3. 在项目根目录下执行 npm link loader-name，把第2步注册到全局的本地 Npm 模块链接到项目的 node_modules 下，其中的 loader-name 是指在第1步中的 package.json 文件中配置的模块名称；

链接好 Loader 到项目后你就可以像使用一个真正的 Npm 模块一样使用本地的 Loader 了。

3.3.5.2. ResolveLoader

为了让 Webpack 加载放在本地项目中的 Loader 需要修改 resolveLoader.modules。

假如本地的 Loader 在项目目录中的 `./loaders/loader-name` 中，则需要如下配置：

 复制代码

```
1 module.exports = {
2   resolveLoader:{
3     // 去哪些目录下寻找 Loader，有先后顺序之分
4     modules: ['node_modules', './loaders/'],
5   }
6 }
```

加上以上配置后，Webpack 会先去 `node_modules` 项目下寻找 Loader，如果找不到，会再去 `./loaders/` 目录下寻找。

3.3.6. 实战

创建名为 `comment-require-loader`，作用是把 JavaScript 代码中的注释语法

 复制代码

```
1 // @require '../style/index.css'
2 转为
3 require('../style/index.css');
```

该 Loader 的使用场景是去正确加载针对 [Fis3](#) 编写的 JavaScript，这些 JavaScript 中存在通过注释的方式加载依赖的 CSS 文件。

该 Loader 的使用方法如下：

 复制代码

```
1 module.exports = {
2   module: {
3     rules: [
4       {
5         test: /\.js$/,
6         use: ['comment-require-loader'],
7         // 针对采用了 fis3 CSS 导入语法的 JavaScript 文件通过 comment-require-l
8         // oader 去转换
9         include: [path.resolve(__dirname, 'node_modules/imui')]
10      }
11    ]
12  };
13 }
```

具体实现时：

 复制代码

```
1 function replace(source) {
2     // 使用正则把 // @require '../style/index.css' 转换成 require('../style/index.css');
3     return source.replace(/(\\\/ *@require) +(('|").+('|").)*\/, 'require ($2);');
4 }
5
6 module.exports = function (content) {
7     return replace(content);
8 };
```

3.4. 编写plugin

Webpack 通过 Plugin 机制让其更加灵活，以适应各种应用场景。在 Webpack 运行的生命周期中会广播许多事件，Plugin 可以监听这些事件，在合适的时机通过 Webpack 提供的 API 改变输出结果。

一个最基础的 Plugin 的代码是这样的：

 复制代码

```
1 class BasicPlugin{
2     // 在构造函数中获取用户给该插件传入的配置
3     constructor(options){
4     }
5
6     // Webpack 会调用 BasicPlugin 实例的 apply 方法给插件实例传入 compiler 对象
7     apply(compiler){
8         compiler.plugin('compilation',function(compilation) {
9             })
10    }
11 }
12
13 // 导出 Plugin
14 module.exports = BasicPlugin;
```

在使用这个 Plugin 时，相关配置代码如下：


```
1  const BasicPlugin = require('./BasicPlugin.js');
2  module.export = {
3    plugins:[
4      new BasicPlugin(options),
5    ]
6  }
```

Webpack 启动后，在读取配置的过程中会先执行 `new BasicPlugin(options)` 初始化一个 `BasicPlugin` 获得其实例。在初始化 `compiler` 对象后，再调用 `basicPlugin.apply(compiler)` 给插件实例传入 `compiler` 对象。插件实例在获取到 `compiler` 对象后，就可以通过 `compiler.plugin(事件名称, 回调函数)` 监听到 Webpack 广播出来的事件。并且可以通过 `compiler` 对象去操作 Webpack。通过以上最简单的 Plugin 相信你大概明白了 Plugin 的工作原理，但实际开发中还有很多细节需要注意：

3.4.1. Compiler 和 Compilation

在开发 Plugin 时最常用的两个对象就是 `Compiler` 和 `Compilation`，它们是 Plugin 和 Webpack 之间的桥梁。`Compiler` 和 `Compilation` 的含义如下：

- `Compiler` 对象包含了 Webpack 环境所有的配置信息，包含 `options`，`loaders`，`plugins` 这些信息，这个对象在 Webpack 启动时候被实例化，它是全局唯一的，可以简单地把它理解为 Webpack 实例；
- `Compilation` 对象包含了当前的模块资源、编译生成资源、变化的文件等。当 Webpack 以开发模式运行时，每当检测到一个文件变化，一次新的 `Compilation` 将被创建。`Compilation` 对象也提供了很多事件回调供插件做扩展。通过 `Compilation` 也能读取到 `Compiler` 对象；

`Compiler` 和 `Compilation` 的区别在于：`Compiler` 代表了整个 Webpack 从启动到关闭的生命周期，而 `Compilation` 只是代表了一次新的编译。

3.4.2. 事件流

Webpack 就像一条生产线，要经过一系列处理流程后才能将源文件转换成输出结果。这条生产线上的每个处理流程的职责都是单一的，多个流程之间存在依赖关系，只有完成当前处理后才能交给下一个流程去处理。插件就像是一个插入到生产线中的一个功能，在特定的时机对生产线上的资源做处理。

Webpack 通过 `Tapable` 来组织这条复杂的生产线。Webpack 在运行过程中会广播事件，插件只需要监听它所关心的事件，就能加入到这条生产线中，去改变生产线的运作。Webpack 的事件流机制保证了插件的有序性，使得整个系统扩展性很好。

Webpack 的事件流机制应用了观察者模式，和 Node.js 中的 EventEmitter 非常相似。Compiler 和 Compilation 都继承自 Tapable，可以直接在 Compiler 和 Compilation 对象上广播和监听事件，方法如下：

 复制代码

```
1  /**
2   * 广播出事件
3   * event-name 为事件名称，注意不要和现有的事件重名
4   * params 为附带的参数
5   */
6   compiler.apply('event-name', params);
7
8  /**
9   * 监听名称为 event-name 的事件，当 event-name 事件发生时，函数就会被执行。
10  * 同时函数中的 params 参数为广播事件时附带的参数。
11  */
12  compiler.plugin('event-name', function(params) {
13
14  });
```

同理，`compilation.apply` 和 `compilation.plugin` 使用方法和上面一致。

在开发插件时，你可能会不知道该如何下手，因为你不知道该监听哪个事件才能完成任务。

在开发插件时，还需要注意以下两点：

- 只要能拿到 Compiler 或 Compilation 对象，就能广播出新的事件，所以在新开发的插件中也能广播出事件，给其它插件监听使用。
- 传给每个插件的 Compiler 和 Compilation 对象都是同一个引用。也就是说在一个插件中修改了 Compiler 或 Compilation 对象上的属性，会影响到后面的插件。
- 有些事件是异步的，这些异步的事件会附带两个参数，第二个参数为回调函数，在插件处理完任务时需要调用回调函数通知 Webpack，才会进入下一处理流程。例如：

 复制代码

```
1  compiler.plugin('emit', function(compilation, callback) {
2      // 支持处理逻辑
3
4      // 处理完毕后执行 callback 以通知 Webpack
5      // 如果不执行 callback，运行流程将会一直卡在这不往下执行
6      callback();
7  });
```

3.4.3. 常用API

插件可以用来修改输出文件、增加输出文件、甚至可以提升 Webpack 性能、等等，总之插件通过调用 Webpack 提供的 API 能完成很多事情。由于 Webpack 提供的 API 非常多，有很多 API 很少用的上，又加上篇幅有限，下面来介绍一些常用的 API：

3.4.3.1. 读取输出资源、代码块、模块及其依赖

有些插件可能需要读取 Webpack 的处理结果，例如输出资源、代码块、模块及其依赖，以便做下一步处理。

在 emit 事件发生时，代表源文件的转换和组装已经完成，在这里可以读取到最终将输出的资源、代码块、模块及其依赖，并且可以修改输出资源的内容。插件代码如下：

```

1  class Plugin {
2    apply(compiler) {
3      compiler.plugin('emit', function (compilation, callback) {
4        // compilation.chunks 存放所有代码块，是一个数组
5        compilation.chunks.forEach(function (chunk) {
6          // chunk 代表一个代码块
7          // 代码块由多个模块组成，通过 chunk.forEachModule 能读取组成代码块的每个模
            块
8          chunk.forEachModule(function (module) {
9            // module 代表一个模块
10           // module.fileDependencies 存放当前模块的所有依赖的文件路径，是一个数组
11           module.fileDependencies.forEach(function (filepath) {
12             });
13           });
14
15           // Webpack 会根据 Chunk 去生成输出的文件资源，每个 Chunk 都对应一个及其以
            上的输出文件
16           // 例如在 Chunk 中包含了 CSS 模块并且使用了 ExtractTextPlugin 时，
17           // 该 Chunk 就会生成 .js 和 .css 两个文件
18           chunk.files.forEach(function (filename) {
19             // compilation.assets 存放当前所有即将输出的资源
20             // 调用一个输出资源的 source() 方法能获取到输出资源的内容
21             let source = compilation.assets[filename].source();
22             });
23           });
24
25           // 这是一个异步事件，要记得调用 callback 通知 Webpack 本次事件监听处理结束。
26           // 如果忘记了调用 callback，Webpack 将一直卡在这里而不会往后执行。
27           callback();
28         })
29       }
30     }

```

3.4.3.2. 监听文件变化

在开发插件时经常需要知道是哪个文件发生变化导致了新的 Compilation，为此可以使用如下代码：

```

1 // 当依赖的文件发生变化时会触发 watch-run 事件
2 compiler.plugin('watch-run', (watching, callback) => {
3     // 获取发生变化的文件列表
4     const changedFiles = watching.compiler.watchFileSystem.watcher.mtimes;
5     // changedFiles 格式为键值对，键为发生变化的文件路径。
6     if (changedFiles[filePath] !== undefined) {
7         // filePath 对应的文件发生了变化
8     }
9     callback();
10 });

```

默认情况下 Webpack 只会监视入口和其依赖的模块是否发生变化，在有些情况下项目可能需要引入新的文件，例如引入一个 HTML 文件。由于 JavaScript 文件不会去导入 HTML 文件，Webpack 就不会监听 HTML 文件的变化，编辑 HTML 文件时就不会重新触发新的 Compilation。为了监听 HTML 文件的变化，我们需要把 HTML 文件加入到依赖列表中，为此可以使用如下代码：

```

1 compiler.plugin('after-compile', (compilation, callback) => {
2     // 把 HTML 文件添加到文件依赖列表，好让 Webpack 去监听 HTML 模块文件，在 HTML 模版
    文件发生变化时重新启动一次编译
3     compilation.fileDependencies.push(filePath);
4     callback();
5 });

```

3.4.3.3. 修改输出资源

有些场景下插件需要修改、增加、删除输出的资源，要做到这点需要监听 emit 事件，因为发生 emit 事件时所有模块的转换和代码块对应的文件已经生成好，需要输出的资源即将输出，因此 emit 事件是修改 Webpack 输出资源的最后时机。

所有需要输出的资源会存放在 compilation.assets 中，compilation.assets 是一个键值对，键为需要输出的文件名称，值为文件对应的内容。

设置 compilation.assets 的代码如下：

```

1  compiler.plugin('emit', (compilation, callback) => {
2    // 设置名称为 fileName 的输出资源
3    compilation.assets[fileName] = {
4      // 返回文件内容
5      source: () => {
6        // fileContent 既可以是代表文本文件的字符串，也可以是代表二进制文件的 Buffer
7        return fileContent;
8      },
9      // 返回文件大小
10     size: () => {
11       return Buffer.byteLength(fileContent, 'utf8');
12     }
13   };
14   callback();
15 });

```

读取 compilation.assets 的代码如下：

```

1  compiler.plugin('emit', (compilation, callback) => {
2    // 读取名称为 fileName 的输出资源
3    const asset = compilation.assets[fileName];
4    // 获取输出资源的内容
5    asset.source();
6    // 获取输出资源的文件大小
7    asset.size();
8    callback();
9  });

```

3.4.4. 判断webpack使用哪些插件

在开发一个插件时可能需要根据当前配置是否使用了其它某个插件而做下一步决定，因此需要读取 Webpack 当前的插件配置情况。以判断当前是否使用了 ExtractTextPlugin 为例，可以使用如下代码：


```

1 // 判断当前配置使用使用了 ExtractTextPlugin,
2 // compiler 参数即为 Webpack 在 apply(compiler) 中传入的参数
3 function hasExtractTextPlugin(compiler) {
4   // 当前配置所有使用的插件列表
5   const plugins = compiler.options.plugins;
6   // 去 plugins 中寻找有没有 ExtractTextPlugin 的实例
7   return plugins.find(plugin=>plugin.__proto__.constructor === ExtractTextP
  lugin) !== null;
8 }

```

3.4.5. 实战

实现一个插件，名叫 EndWebpackPlugin，作用是在 Webpack 即将退出时再附加一些额外的操作，例如在 Webpack 成功编译和输出了文件后执行发布操作把输出的文件上传到服务器。同时该插件还能区分 Webpack 构建是否执行成功。使用该插件时方法如下：

```

1 module.exports = {
2   plugins:[
3     // 在初始化 EndWebpackPlugin 时传入了两个参数，分别是在成功时的回调函数和失败时
    的回调函数；
4     new EndWebpackPlugin(() => {
5       // Webpack 构建成功，并且文件输出了后会执行到这里，在这里可以做发布文件操作
6     }, (err) => {
7       // Webpack 构建失败，err 是导致错误的原因
8       console.error(err);
9     })
10  ]
11 }

```

要实现该插件，需要借助两个事件：

- done：在成功构建并且输出了文件后，Webpack 即将退出时发生；
- failed：在构建出现异常导致构建失败，Webpack 即将退出时发生；

实现该插件非常简单，完整代码如下：

```
1  class EndWebpackPlugin {
2
3  constructor(doneCallback, failCallback) {
4    // 存下在构造函数中传入的回调函数
5    this.doneCallback = doneCallback;
6    this.failCallback = failCallback;
7  }
8
9  apply(compiler) {
10    compiler.plugin('done', (stats) => {
11      // 在 done 事件中回调 doneCallback
12      this.doneCallback(stats);
13    });
14    compiler.plugin('failed', (err) => {
15      // 在 failed 事件中回调 failCallback
16      this.failCallback(err);
17    });
18  }
19 }
20 // 导出插件
21 module.exports = EndWebpackPlugin;
```

从开发这个插件可以看出，找到合适的事件点去完成功能在开发插件时显得尤为重要。

4. webpack补充

4.1. 常见的Loaders

4.1.1. 加载文件

- **raw-loader**: 把文本文件的内容加载到代码中；
- **file-loader**: 把文件输出到一个文件夹中，在代码中通过相对 URL 去引用输出的文件；
- **url-loader**: 和 file-loader 类似，但是能在文件很小的情况下以 base64 的方式把文件内容注入到代码中去；
- **source-map-loader**: 加载额外的 Source Map 文件，以方便断点调试；
- **svg-inline-loader**: 把压缩后的 SVG 内容注入到代码中；
- **node-loader**: 加载 Node.js 原生模块 .node 文件；

- [image-loader](#): 加载并且压缩图片文件;
- [json-loader](#): 加载 JSON 文件;
- [yaml-loader](#): 加载 YAML 文件;

4.1.2. 编译模版

- [pug-loader](#): 把 Pug 模版转换成 JavaScript 函数返回;
- [handlebars-loader](#): 把 Handlebars 模版编译成函数返回;
- [ejs-loader](#): 把 EJS 模版编译成函数返回;
- [haml-loader](#): 把 HAML 代码转换成 HTML;
- [markdown-loader](#): 把 Markdown 文件转换成 HTML;

4.1.3. 转换脚本语言

- [babel-loader](#): 把 ES6 转换成 ES5;
- [ts-loader](#): 把 TypeScript 转换成 JavaScript;
- [awesome-typescript-loader](#): 把 TypeScript 转换成 JavaScript, 性能要比 ts-loader 好;
- [coffee-loader](#): 把 CoffeeScript 转换成 JavaScript;

4.1.4. 转换样式文件

- [css-loader](#): 加载 CSS, 支持模块化、压缩、文件导入等特性;
- [style-loader](#): 把 CSS 代码注入到 JavaScript 中, 通过 DOM 操作去加载 CSS;
- [sass-loader](#): 把 SCSS/SASS 代码转换成 CSS;
- [postcss-loader](#): 扩展 CSS 语法, 使用下一代 CSS;
- [less-loader](#): 把 Less 代码转换成 CSS 代码;
- [stylus-loader](#): 把 Stylus 代码转换成 CSS 代码;

4.1.5. 检查代码

- [eslint-loader](#): 通过 ESLint 检查 JavaScript 代码;
- [tslint-loader](#): 通过 TSLint 检查 TypeScript 代码;
- [mocha-loader](#): 加载 Mocha 测试用例代码;
- [coverjs-loader](#): 计算测试覆盖率;

4.1.6. 其它

- [vue-loader](#): 加载 Vue.js 单文件组件;
- [i18n-loader](#): 加载多语言版本, 支持国际化;
- [ignore-loader](#): 忽略掉部分文件;
- [ui-component-loader](#): 按需加载 UI 组件库, 例如在使用 antd UI 组件库时, 不会因为只用到了 Button 组件而打包进所有的组件;

4.2. 常见的Plugins

4.2.1. 用于修改行为

- [define-plugin](#): 定义环境变量;
- [context-replacement-plugin](#): 修改 require 语句在寻找文件时的默认行为;
- [ignore-plugin](#): 用于忽略部分文件;

4.2.2. 用于优化

- [commons-chunk-plugin](#): 提取公共代码;
- [extract-text-webpack-plugin](#): 提取 JavaScript 中的 CSS 代码到单独的文件中;
- [prepack-webpack-plugin](#): 通过 Facebook 的 Prepack 优化输出的 JavaScript 代码性能;
- [uglifyjs-webpack-plugin](#): 通过 UglifyES 压缩 ES6 代码;
- [webpack-parallel-uglify-plugin](#): 多进程执行 UglifyJS 代码压缩, 提升构建速度;
- [imagemin-webpack-plugin](#): 压缩图片文件;
- [webpack-spritesmith](#): 用插件制作雪碧图;
- [ModuleConcatenationPlugin](#): 开启 Webpack Scope Hoisting 功能;
- [dll-plugin](#): 借鉴 DDL 的思想大幅度提升构建速度;
- [hot-module-replacement-plugin](#): 开启模块热替换功能;

4.2.3. 其它

- [serviceworker-webpack-plugin](#): 给网页应用增加离线缓存功能;
- [stylelint-webpack-plugin](#): 集成 stylelint 到项目;
- [i18n-webpack-plugin](#): 给你的网页支持国际化;

- [provide-plugin](#): 从环境中提供的全局变量中加载模块，而不用导入对应的文件；
- [web-webpack-plugin](#): 方便的为单页应用输出 HTML，比 [html-webpack-plugin](#) 好用；

4.3. 其他webpack学习资料

- [Webpack 官网](#): 官方网站，涵盖入门教程，以及详细 API 文档；
- [Webpack 中文文档](#): 翻译自官方网站；
- [Webpack 官方博客](#): 在 Medium 上的 Webpack 官方博客，能看到最新特性和未来计划；
- [SurviveJS – Webpack](#): 来自 SurviveJS 的 Webpack 英文电子书，从入门到精通；
- [webpack-examples](#): 来自 Webpack 官方的案例集合；
- [webpack-demos](#): 来自阮一峰的 Webpack 案例集合；
- [awesome-webpack](#): Webpack 优秀周边生态收集；
- [awesome-webpack-cn](#): Webpack 优秀中文文章；